

Lambda: A 4 mm² Open-Source Transformer Inference ASIC with Streaming Per-Channel KV Compression and Adaptive-Precision KV

Alan Schwartz

Longhorn Silicon Project

The University of Texas at Austin

aschwartz0408@utexas.edu

<https://github.com/LonghornSilicon>

Abstract

We present **Lambda**, a 4 mm² standalone transformer-decode ASIC at TSMC N16FFC, designed for tape-out via the IMEC / Europractice mini@sic 2.0 academic shuttle program. Lambda runs W4A8 transformer models up to 1.5 B parameters—validated on Qwen2-1.5B—at 6–8 tokens/s decode in a 2.6 W typical envelope, on a single LPDDR5X-8533 x16 channel sustaining 12 GB/s. Three architectural contributions distinguish the chip. (1) We implement ChannelQuant, a streaming per-channel KV-compression codec, in silicon: per-channel INT4 keys (grouped, $G=128$, with per-channel FP16 scales), per-token INT4 values, and a static top- k ($k=2$) FP16 outlier-channel lane selected by a calibrated ROM mask. The recipe follows KIVI [1] and KVQuant [2]; our contribution is the streaming silicon implementation. Measured compression is $\approx 3.8\times$ versus FP16 at ≈ 4 bits per value, near-lossless (HellaSwag acc_norm within ≈ 0.4 – 0.8 pt of FP16 at the CQ-4+ tier on Qwen2-0.5B/1.5B). (2) A single shared FP16 compute unit (scale / quantize / dequantize) is serialized across the head-dimension channels—a single divide cone—and keys are dequantized per-channel (INT4 $\cdot$ FP16) before the score matmul; there is no compressed-domain read path. (3) We introduce a Token Importance Unit (TIU): a 0.03 mm² block that accumulates per-block attention entropy and drives both heavy-hitter eviction and adaptive per-block ChannelQuant-tier selection in the KV cache. The complete chip integrates seven functional units under an ACU/MSC/LSU/TIU/HIF top-level hierarchy and presents a PCIe Gen3 x1 host interface on an M.2 2280 form factor. We document the design-space exploration that led to the 4 mm² target, the LPDDR5X-vs-LPDDR4X PHY tradeoff that gates the model-class commitment, and an explicit pre-RTL audit of the spec. Physical PPA numbers for the KV-compression block at N16FFC (area, power, $F_{\max}$) are pending re-measurement for ChannelQuant. Lambda is, to our knowledge, the first open-source academic standalone transformer accelerator targeting the small-model (≤ 1.5 B-parameter) on-device deployment regime.

Index Terms—LLM inference accelerator, KV-cache compression, ChannelQuant, per-channel quantization, FlashAttention, paged attention, adaptive precision, TSMC N16FFC, academic ASIC, IMEC mini@sic

I. INTRODUCTION

The small-model transformer—up to a few billion parameters—has become the practical on-device deployment regime for large language models in 2025–2026. Qwen2-

1.5B [3], Llama-3.2-1B/3B [4], Mistral-NeMo [5], Phi-3.5-mini [6], and Gemma-2-2B all target this band; Apple’s recently-published foundation-language-model family [7], Google Gemini Nano, and Microsoft Copilot+ all deploy at similar scales. The accelerators that serve these workloads in production—Apple’s Neural Engine, Qualcomm’s Hexagon NPU—are proprietary integrated SoCs whose internal architectures are not publicly documented. Modern GPU accelerators [8]–[10] serve the workload at the data-center scale but at die areas and power envelopes 10^2 – 10^3 times Lambda’s. No open-source academic standalone accelerator at this model class and process node currently exists.

Lambda fills that gap. We present a 4 mm² ASIC at TSMC N16FFC, designed for tape-out through the IMEC / Europractice mini@sic 2.0 academic shuttle program [11] at a shuttle cost of approximately \$60–100 K. The chip runs a W4A8 transformer up to 1.5 B parameters (validated on Qwen2-1.5B) at 6–8 decode tokens per second in a 2.6 W typical envelope, holding 4–8 GB of weights in an off-chip LPDDR5X-8533 x16 package and exposing host I/O over a PCIe Gen3 x1 link on an M.2 2280 carrier card.

Three architectural commitments differentiate Lambda from prior open-source accelerator designs:

(1) **Streaming per-channel KV compression in silicon (ChannelQuant)**. Lambda’s KV-compression engine (KVE) implements *ChannelQuant*: per-channel INT4 keys (grouped, $G=128$, with per-channel FP16 scales), per-token INT4 values (INT8 in the CQ-8 tier), and a static top- k ($k=2$) FP16 outlier-channel lane selected by a calibrated ROM mask. Keys and values share a single per-channel SRAM record and one serialized FP16 scale/quantize/dequantize unit. The recipe follows KIVI [1] and KVQuant [2]; our contribution is the streaming silicon implementation. Measured compression is $\approx 3.8\times$ versus FP16 at ≈ 4 bits per value (4.13–4.38 depending on head dimension D), near-lossless: HellaSwag acc_norm falls within ≈ 0.4 – 0.8 pt of FP16 at the CQ-4+ tier on Qwen2-0.5B/1.5B. The full block RTL is complete through Sky130 sign-off; physical PPA at N16FFC is pending re-measurement.

(2) **Per-channel dequantization before scoring.** The datapath serializes one shared FP16 compute unit across the D head-dimension channels (a single divide cone), and keys are reconstructed per-channel ($\text{INT4} \cdot \text{FP16}$, with FP16 replay for the $k=2$ outlier channels) *before* the attention dot product. Lambda’s matrix engine (MatE) therefore keeps weight and FFN GEMMs at $\text{INT8} \times \text{INT4}$ and scores $Q \cdot K^\top$ on the dequantized K ($\text{INT8 } Q \times \text{dequantized FP16 } K$); there is no compressed-domain / raw-index read path, and the KV codec itself needs no fallback to FP16. FP16 multiplication in MatE is confined to the per-tile $P \cdot V$ escape (§IV, selected by the ACU precision controller); the remaining FP16 work is in VecU (softmax, RMSNorm, RoPE, SiLU) and the KVE’s shared scale unit.

(3) **Token Importance Unit (TIU) for adaptive precision.** Inspired by the entropy-driven KV quantization analysis of [12], we introduce a small (0.03 mm^2) on-die accumulator that tracks the cumulative softmax weight per 16-token KV block. The TIU output drives two consumer paths: a heavy-hitter eviction policy in the memory subsystem controller (H2O-style retention [13]) and a per-block ChannelQuant-tier selector that keeps high-importance blocks at a higher tier (CQ-4+ or CQ-8) and demotes low-importance blocks to CQ-4. To our knowledge, this is the first silicon implementation of adaptive-precision KV cache management; published prior work is software-only.

The remainder of this paper develops these contributions in detail. Section II surveys the prior art the design is grounded in. Section III describes Lambda’s seven functional blocks and their top-level grouping. Section IV traces the decode token data flow and the quantization decisions that shape it. Section V reports the area and power budgets, including a pre-RTL audit that corrected eight numerical inconsistencies in the original spec. Section VI characterizes Lambda’s coverage across the candidate-model space and quantifies the load-bearing LPDDR5X-vs-LPDDR4X PHY tradeoff. Section VII discusses what we deliberately excluded and why.

II. BACKGROUND AND RELATED WORK

Lambda integrates four lines of recent research into a single chip target.

Online attention. FlashAttention [14]–[16] reformulates softmax as an online tiled reduction with per-row running maxima m_i and running sums-of-exponentials ℓ_i . This eliminates materialization of the score matrix and reduces the activation-memory pressure that dominates long-context attention. Lambda’s vector unit (VecU) implements the FlashAttention-3 online softmax algorithm in microcode; the running state per row is two FP16 scalars plus the running output accumulator O_i . VecU also implements the rotary positional embedding scheme (RoPE) [17], RMSNorm [18], and the SwiGLU gating used by the Llama-3 family [19] as microcoded loops.

Paged KV management. vLLM’s PagedAttention [20] treats the KV cache as a virtual address space with fixed-size physical blocks, indexed by a per-sequence block table. The pattern enables zero-fragmentation memory use and dynamic eviction. Lambda’s memory subsystem controller (MSC) implements

this in silicon as a 128-entry content-addressable block table with 16-token blocks, the silicon analog of vLLM’s logical-to-physical mapping. FlashInfer [21] extends the paged abstraction to a composable taxonomy of attention patterns (dense paged, sparse structured, ragged, composite); Lambda’s MSC exposes a CSR-selectable mode covering this taxonomy.

Weight and activation quantization. W4A8 has emerged as the production sweet spot for small-LLM inference [22]: 4-bit weights (via GPTQ [23] or AWQ [24]) and 8-bit per-token dynamic activations (via SmoothQuant [25]) preserve quality on the small-model class within a few MMLU points of FP16. Lambda’s MatE multiplies INT8 activations against INT4 weights as its primary path.

KV-cache compression. A growing body of work [1], [2], [26]–[29] compresses KV caches via uniform quantization, per-channel/per-token quantization, rotation, codebook quantization, or per-token mixed precision. KIVI [1] establishes the asymmetric recipe Lambda adopts—*per-channel* quantization for keys (each channel gets its own scale, so a large-magnitude channel does not coarsen the whole tile) and *per-token* quantization for values—and KVQuant [2] adds a dense-and-sparse split that keeps a small set of outlier channels in high precision. Lambda’s ChannelQuant codec follows this recipe: per-channel INT4 keys (grouped $G=128$), per-token INT4 values, and a static top- k ($k=2$) FP16 outlier-channel lane. Rotation-codebook methods such as TurboQuant [29] are a different point in the design space (Walsh–Hadamard rotation plus a Lloyd–Max scalar codebook); we cite them as prior work but do not implement them. Because keys are dequantized per-channel ($\text{INT4} \cdot \text{FP16}$) before scoring, Lambda’s codec needs no compressed-domain read trick, and $Q \cdot K^\top$ scores on dequantized K with no fallback to FP16 in the KV read path. FP16 in MatE is confined to the per-tile $P \cdot V$ escape (§IV): the $P \cdot V$ matmul does adopt a per-tile INT8/FP16 route in the manner of adaptive-precision designs [12], [30], while weight/FFN GEMMs stay $\text{INT8} \times \text{INT4}$.

Attention-weight-driven eviction and adaptive precision. H2O [13], Scissorhands [31], TOVA [32], and the StreamingLLM “attention-sink” phenomenon [33] all show that retaining tokens whose cumulative attention weight exceeds a threshold (and evicting the rest) preserves long-context quality at a fraction of the KV-cache footprint. The recent adaptive-precision-KV work [12] extends this idea to bit-width allocation: high-attention tokens stay at higher precision, low-attention tokens demote. Lambda’s TIU is the silicon expression of this idea at per-block granularity (16 tokens per block, matching the PagedAttention block size).

III. ARCHITECTURE

Figure 1 shows Lambda’s top-level decomposition. Seven functional blocks are grouped under five top-level units—ACU (Attention Compute Unit), MSC (Memory Subsystem Controller), LSU (Layer Sequencer), TIU (Token Importance Unit), and HIF (Host Interface)—plus 0.8 MB of on-die SRAM in four banks and a vendor-IP LPDDR5X x16 PHY. The ACU is itself a composite of three blocks (MatE, VecU, KVE) that

share the compute fabric of the chip; the naming convention follows [30] while preserving the internal block structure for high-level synthesis (HLS) modularity. We discuss each block in turn.

A. Matrix Engine (MatE)

MatE is an 8×8 weight-stationary systolic array of $64 \text{ INT8} \times \text{INT4}$ processing elements (PEs). At 1 GHz, peak throughput is 128 GOPS; sustained throughput at 60% utilization is 76.8 GOPS—a figure that is, in the bandwidth-bound regime, $1.6 \times$ the throughput the chip needs at the largest model size. The factor is constant across model sizes in this regime because the GOPS demand scales with bandwidth (not with model size); we found and corrected an earlier draft claim of “ $3\text{--}5 \times$ headroom” as a derivation error during the spec audit.

Two dataflow modes are CSR-selectable:

Weight-stationary for the GEMMs that dominate transformer compute: Q/K/V projections, attention output projection, FFN up- and down-projections, and the final logits projection. Weights pin once per tile; activations stream from the activation buffer; products accumulate down the columns.

Output-stationary for the $Q \cdot K^\top$ attention scoring step. Q vectors pin for the duration of the attention tile; K values stream from the kv_scratchpad. K is stored in ChannelQuant form (per-channel INT4 codes plus per-channel FP16 scales); the KVE reconstructs it per-channel ($\text{INT4} \cdot \text{FP16}$, with FP16 replay for the $k=2$ outlier channels) *before* it enters the array, so MatE scores against dequantized K—there is no compressed-domain / raw-index read path. The products feed an INT24 reduction along the head-dimension axis. We selected INT24 (rather than the INT16 the original spec contained) after analysis showed the INT16 width saturates on the FFN reductions of the earlier target-model dims; the INT16 width survives only as the partial-product register inside each PE, following the per-PE/per-column accumulator split of Jouppi’s TPU [34], [35]. The exact accumulator margin for the Qwen2-1.5B target is pending re-derivation.

B. Vector Unit (VecU)

VecU is eight FP16/BF16 SIMD lanes sharing a 1 K-instruction microcode RAM and three 64-entry transcendental LUTs ($\exp$, rsqrt , sigmoid) with linear interpolation. The unit handles every non-GEMM operation in the transformer: FlashAttention-3 online softmax, rotary positional embedding (RoPE) pair-rotation, RMSNorm, SiLU/GELU, residual addition, and sampling. We chose programmable SIMD over fixed-function blocks for verification economy—one microcoded block has a smaller verification surface than four fixed-function blocks.

The online softmax microcode is the most consequential program. Per attention row, VecU maintains a triple (m_i, ℓ_i, O_i) : running max, running sum-of-exponentials, and running output accumulator. As each tile of scores arrives, VecU computes the new max, rescales the running accumulators by $e^{m_{\text{old}} - m_{\text{new}}}$, exponentiates the tile via the $\exp$ LUT, and accumulates into ℓ_i and into O_i weighted by the corresponding V tile. The 8 lanes

parallelize across rows; tile size is 32–64 scores; approximately 32 microinstructions execute per tile.

C. KV Cache Engine (KVE) — ChannelQuant

The KVE is the headline research IP: a streaming silicon implementation of *ChannelQuant*. It compresses keys and values as MatE produces them and reconstructs them per-channel on the read path. For keys, the KVE buffers a group of $G=128$ keys, takes the per-channel absolute maximum over the group, derives D per-channel FP16 scales, and quantizes each channel to INT4; the static top- k ($k=2$) outlier channels—selected by a calibrated ROM mask—bypass INT4 and are held in FP16. Values are quantized per token to INT4 (INT8 in the CQ-8 tier). Keys and values live in a unified per-channel SRAM record {tag, $D \times \text{FP16}$ scale, $D \times \text{INT4}$ code}. Decompression is per-channel $\text{INT4} \cdot \text{FP16}$, with FP16 replay for the outlier channels. Measured cost is ≈ 4 bits per value (4.13–4.38 depending on head dimension D), i.e. $\approx 3.8 \times$ versus FP16, and near-lossless (HellaSwag acc_norm within $\approx 0.4\text{--}0.8$ pt of FP16 at CQ-4+ on Qwen2-0.5B/1.5B). The recipe follows KIVI [1] and KVQuant [2]; our contribution is the streaming silicon datapath.

The datapath serializes a *single* shared FP16 compute unit (scale, quantize, dequantize) across the D head-dimension channels—one divide cone reused on both the compress and decompress paths—rather than a wide parallel array. Three tiers plus a bypass are CSR-selectable: **CQ-8** (per-token INT8 K and V), **CQ-4** (per-channel INT4 K, per-token INT4 V; the primary tier), **CQ-4+** (CQ-4 plus the $k=2$ FP16 outlier channels; the near-lossless tier), and an FP16 bypass for ablation. The full block RTL is complete through Sky130 sign-off in the kv-cache-engine repository; the physical PPA of the block at N16FFC (area, power, F_{max}) is pending re-measurement for ChannelQuant.

D. Memory Subsystem Controller (MSC)

MSC integrates four functions in 0.18 mm^2 : the LPDDR5X x16 protocol-side controller (with open-page policy and bank-conflict avoidance), a 4-port SRAM crossbar serving MatE, VecU, KVE, and the host, a 128-entry block table providing the silicon analog of vLLM’s PagedAttention [20] indexing (16 tokens per block, 2 K addressable tokens per session), and a DMA descriptor FSM. Sparse-blocked attention is exposed as a CSR-selectable mode (`attention_pattern_modes`) following the FlashInfer taxonomy [21]: dense paged, sliding window, sparse blocked, local+global hybrid (the Gemma-2 layout).

MSC is single-session by design. We exclude continuous batching (multi-session state), tier-3 host-DRAM eviction (we have no PCIe-to-host swap path on a 4 mm^2 die), and cross-session prefix sharing. These omissions are deliberate: the chip is sized for batch-1 on-device inference, not multi-tenant serving.

E. Layer Sequencer (LSU)

The LSU is a minimal three-stage in-order RISC: 32-bit fixed-width instruction format, 32 instructions, $16 \times 32\text{-bit}$

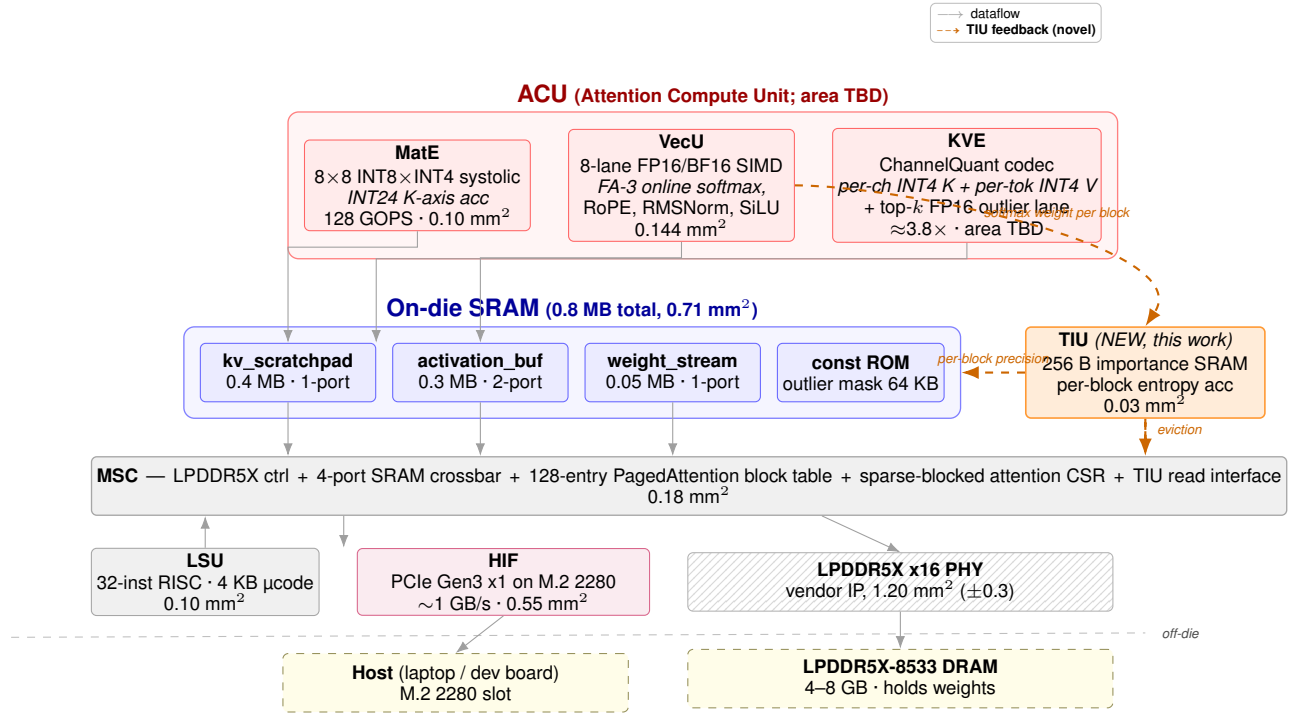


Fig. 1. Lambda’s top-level architecture, drawn full page width to keep the dataflow legible. The chip’s seven functional blocks are grouped under five top-level units. The **ACU (Attention Compute Unit, red)** contains the three compute blocks that share the GEMM and SIMD fabric (MatE, VecU, KVE). The four on-die SRAM banks (blue, 0.8 MB total) sit between the compute fabric and the memory controller. The **MSC** manages LPDDR access, the 128-entry PagedAttention block table, sparse-blocked attention CSR modes, and the TIU read interface. The **LSU** walks a pre-compiled 4 KB microcode schedule. The **TIU (orange, novel to this work)** accumulates per-block attention entropy from VecU’s softmax broadcasts and drives two feedback paths drawn in dashed orange: per-block ChannelQuant-tier selection to the KVE, and heavy-hitter-driven eviction to MSC. The **HIF** is a PCIe Gen3 x1 endpoint on an M.2 2280 carrier; the LPDDR5X x16 PHY drives the off-die LPDDR5X package. Areas shown are at the PHY mid-case (1.20 mm²); the KVE block area is pending re-measurement for ChannelQuant.

general-purpose registers, 4 KB of microcode RAM holding the entire pre-compiled per-layer schedule, single-issue across three dispatch lanes (scalar, vector to ACU, DMA to MSC). No branch predictor, no out-of-order machinery, no cache hierarchy: transformer decode is structurally identical layer to layer, and a static schedule walked deterministically suffices. The host loads roughly 3 K microcode instructions at boot; the chip executes the same schedule forever. The footprint is 0.10 mm².

F. Token Importance Unit (TIU)

TIU is a 0.03 mm² block introduced in this work. It maintains a 256-byte importance SRAM organized as 128 entries of 16 bits each (one entry per PagedAttention block). Update path: VecU broadcasts the per-block cumulative softmax weight of the current attention pass; the TIU adds to the corresponding importance register. Consumer paths: (i) MSC queries the lowest-importance block when the kv_scratchpad approaches capacity (heavy-hitter eviction, H2O-style [13]), and (ii) the KVE queries the per-block importance bucket when re-compressing an evicted-and-recalled block, demoting low-importance blocks to the CQ-4 tier while keeping high-importance blocks at a higher tier (CQ-4+ or CQ-8).

Four CSR-selectable policies cover the design space: `tiu_off` (pure FIFO eviction, uniform precision),

`tiu_h2o` (heavy-hitter retention, uniform precision), `tiu_streaming_llm` (recent + attention-sink retention), and `tiu_adaptive_precision` (heavy-hitter retention plus per-block adaptive precision). The block is grounded in the adaptive-precision-KV analysis of [12], which we believe has not previously been implemented in silicon.

G. Host Interface (HIF)

HIF is a PCIe Gen3 x1 endpoint (1 GB/s sustained) on the M.2 2280 form factor at the PCB level. The M.2 connector wires four PCIe lanes by specification; Lambda drives only one on-die, and PCIe link training negotiates the link down to x1. Vendor IP (Synopsys DesignWare or Cadence’s PCIe Gen3 PHY at 16 nm) provides the analog and protocol stack; we implement the BAR0 CSR map, the 16-deep doorbell command queue, and the JTAG TAP. JTAG runs on dedicated pins, separate from the PCIe lanes, so post-silicon debug remains accessible before PCIe link is up. Total block area is 0.55 mm², of which the vendor PHY accounts for approximately 0.35 mm².

We initially specified USB-C 2.0 for this interface and revised to PCIe Gen3 x1 mid-design after concluding that the 25-second weight-load latency of USB-C 2.0 was a development-iteration cost we did not want to accept, and that the M.2 form factor better matched the modern inference-card target. The PCIe

vendor IP also has public 16 nm datasheets, in contrast to the LPDDR5X x16 PHY which remains NDA-gated at 16 nm; the PCIe-side risk profile is therefore lower.

IV. DATAFLOW AND QUANTIZATION

We walk one decode token through the pipeline to make the quantization commitments concrete. For continuity with the earlier design iterations we use a 3-billion-parameter model as the illustrative example (Llama-3.2-3B, 28 layers, 8 KV heads of 128 dimensions, 3072 hidden width, FFN width 8192); the canonical target is smaller (Qwen2-1.5B), and the per-layer/per-token *timing* figures in this example are pending re-derivation for that target.

The LSU issues approximately 200 microinstructions per layer. For each layer, the schedule alternates four operand-stationary patterns. **(1) Q/K/V projection:** MatE consumes the activation vector and three weight matrices (streamed from LPDDR via the `weight_stream_buffer`), in weight-stationary mode, producing three 1024-dimensional output vectors. **(2) RoPE rotation:** VecU rotates Q and K pair-wise using the rotary frequency table. **(3) KV compression:** the KVE compresses the new K per-channel (INT4 codes plus D FP16 scales, $k=2$ outlier channels held FP16) and the new V per-token (INT4), writing the unified per-channel record into the `kv_scratchpad`. **(4) Attention scoring:** MatE switches to output-stationary mode and computes $Q \cdot K^\top$ against K that the KVE has *dequantized per-channel* (INT4 · FP16) just ahead of the array. The multiplier path is INT8 (Q) $\times$ dequantized K, and the K-axis reduction is INT24. There is no compressed-domain / raw-index read path.

The scores feed VecU, which executes the FlashAttention-3 online softmax microcode in tiles of 32. Simultaneously, the running output accumulator O_i is updated by multiplying scaled scores against the V values (dequantized per-token by the KVE) through the same MatE path; this $P \cdot V$ matmul is the one place MatE routes precision per tile (INT8 or FP16, §IV-A). When the last tile is consumed, O_i is the attention output. The TIU has been quietly accumulating the per-block cumulative softmax weight throughout this loop; the importance SRAM is now ready for the next eviction or precision-tuning decision.

The post-attention path runs through MatE again (attention output projection, FFN up-projection, FFN down-projection) and VecU (SiLU, residual add, RMSNorm). Compute is bandwidth-bound throughout: the limiting factor is the time required to stream the weights of each linear layer from LPDDR5X, giving a 6–8 tokens-per-second decode cadence for the target model class. (The per-layer and per-token timing figures were derived for the retired 3–5 B example set and are pending re-derivation for the Qwen2-1.5B target.)

A. Adaptive precision: the FP16 $P \cdot V$ path

Adaptive-precision-attention designs [12], [30] route “outlier” attention tiles to an FP16 multiplier and “inlier” tiles to an INT8 multiplier, with a per-tile gating function deciding which path to take. The rationale is that some attention tiles are dominated by a few high-magnitude scores that would lose

precision under INT8. Lambda draws a precise line around where this run-time gate is—and is not—needed.

Where the gate is unnecessary: the KV codec and $Q \cdot K^\top$. ChannelQuant handles KV outliers *structurally at compress-time*, not per-tile at read-time. Because keys are quantized *per channel*, each channel’s dynamic range is captured by its own FP16 scale, so a large-magnitude channel does not force a coarse quantizer on the rest of the tile; the $k=2$ channels that would still degrade INT4—the static outliers selected by the calibrated ROM mask—are simply held in FP16 in the outlier lane. The KVE therefore dequantizes keys per-channel to FP16 *before* the array, and $Q \cdot K^\top$ scores as INT8 (Q) against dequantized K. There is no run-time “outlier tile” to route in the codec or in scoring, and no fallback to FP16 in the KV read path. The empirical evidence is in the codec’s accuracy: HellaSwag `acc_norm` within ≈ 0.4 – 0.8 pt of FP16 at the CQ-4+ tier on Qwen2-0.5B/1.5B, consistent with KIVI [1] and KVQuant [2]. Weight and FFN GEMMs likewise stay INT8 $\times$ INT4 throughout.

Where the gate is needed: $P \cdot V$. The attention-output matmul $P \cdot V$ is a different case. Here the operands are the online-softmax probabilities P —which, on peaked-attention tiles, concentrate almost all their mass on a few positions—times the per-token-dequantized V . A tile whose probability vector is dominated by a handful of large entries loses precision under a single INT8 scale in exactly the way the adaptive-precision analysis predicts, and per-channel KV quantization does nothing to help, because the peaking is a property of the *scores*, not of the V channels. Lambda therefore routes $P \cdot V$ per tile: the ACU precision controller evaluates the entropy-equivalent ratio $\max(|s|) \cdot N > 10 \cdot \sum(|s|)$ over each tile and escalates peaked tiles to the FP16 MAC path, leaving the common (near-uniform) tiles on the INT8 path. The controller and the FP16 MAC-array datapath are the Sky130-signed-off adaptive-precision-attention block, reused verbatim. FP16 multiplication in MatE is thus confined to this per-tile $P \cdot V$ escape; the remaining FP16 work is in VecU (softmax, RMSNorm, RoPE, transcendentals) and the KVE’s shared scale unit. The area and power cost of adding the FP16 mode to the array is pending re-synthesis at N16FFC and is not yet quantified here.

V. IMPLEMENTATION

A. Process, target, and shuttle

Lambda targets TSMC’s N16FFC FinFET process at a logic density of approximately 28.2 MTr/mm² and an HD SRAM density of approximately 1.25 MB/mm², with a core voltage of 0.8 V and a target clock of 1 GHz (800 MHz fallback specified). Tape-out is via the IMEC / Europractice mini@sic 2.0 academic shuttle program [11]; the Muse Semiconductor US-focused TSMC University FinFET path is the documented fallback. Shuttle cost is approximately \$60–100 K at the academic-discount tier; total chip cost including PHY IP licensing (LPDDR5X x16 vendor PHY plus PCIe Gen3 x1 vendor PHY) is approximately \$170–290 K. The design flow is Cadence end-to-end: Stratus HLS for C++-to-RTL synthesis,

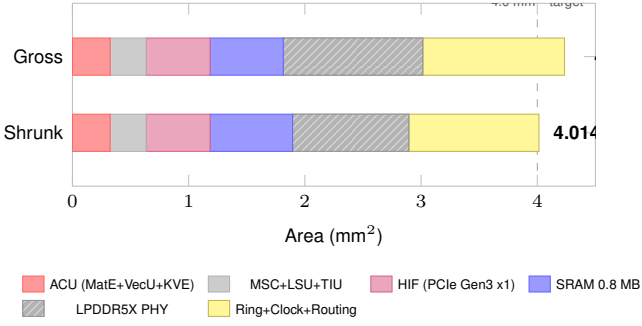


Fig. 2. Area accounting at PHY mid-case (*Gross*, 4.354 mm²) and after the recommended shrink path—vendor PHY at 1.0 mm² best case, I/O ring tightened to 80 μ m, activation buffer trimmed to 0.2 MB—which lands at 4.014 mm², within 0.4% of the 4.0 mm² target. Earlier spec drafts dropped the routing-overhead buffer (0.10 mm²) from the total and claimed 3.974 mm²; the pre-RTL audit corrected this.

Genus for logic synthesis, Innovus for place-and-route (with DREAMPlace [36] explored as an open-source comparison pre-tape-out), Calibre for DRC/LVS, and PrimeTime for static timing signoff. Compiler-side experiments target MLIR [37] as the lowering substrate for the LSU schedule generator. Tool access is bundled with mini@sic registration.

B. Area and power budgets

Table I reports per-block area, verification surface, and risk classification; Figure 2 visualizes the gross-versus-shrunk accounting. The gross total at the PHY mid-case (1.20 mm²) is 4.354 mm², exceeding the 4 mm² target by 0.354 mm². The recommended shrink path—PHY landing at 1.0 mm² vendor best case (−0.20 mm²), I/O ring tightened from 100 μ m to 80 μ m (−0.06 mm²), and the activation buffer shrunk from 0.3 MB to 0.2 MB (−0.08 mm²)—lands at 4.014 mm², within 0.4% of target. The shrink is contingent on the LPDDR5X x16 PHY vendor quote returning at or below 1.0 mm²; we discuss the LPDDR4X fallback in Section VI.

Power decomposes to 2.6 W typical decode and 3.3 W peak prefill at 1 GHz. The LPDDR5X PHY and DRAM together draw 0.72 W (at the 7.5 mW/Gbps planning figure for LPDDR5X-class memory and 96 Gbps sustained). MatE at 50% utilization is 0.32 W (at the 5 W/TOPS heuristic for INT8 systolic at 16 nm), VecU 0.16 W, SRAM 0.16 W, the KVE holds a 0.05 W placeholder (pending re-measurement for ChannelQuant), TIU 0.01 W, MSC+LSU+HIF 0.50 W. Whole-chip leakage at 16 nm at 0.8 V is approximately 0.70 W. The M.2 2280 slot delivers approximately 10 W per spec; we sit at 26% of that envelope at typical decode.

C. Pre-RTL audit

Before any HLS work begins we ran an explicit numerical audit of the spec and corrected eight inconsistencies. We document the audit here because each correction is a load-bearing decision that the rest of the chip integrates over.

KV bytes per token were 2.3 $\times$ understated. The bytes-per-token-per-layer formula in the original spec used $k_h \times d_h \times \text{bpv}/8$, dropping the factor of 2 for K and V separately. We re-derived every capacity number against the correct $k_h \times$

TABLE I
PER-BLOCK AREA, VERIFICATION SURFACE, AND RISK CLASS AT PHY MID-CASE (1.20 mm²). [†]The KVE (CHANNELQUANT) BLOCK AREA AT N16FFC IS PENDING RE-MEASUREMENT; THE GROSS TOTAL HOLDS A 0.08 mm² PLACEHOLDER FOR IT.

Block	Area (mm ²)	Tests	Risk
MatE (8 $\times$ 8 INT8 $\times$ INT4)	0.10	50	medium
VecU (8-lane SIMD)	0.144	25	medium
KVE (ChannelQuant)	TBD [†]	60	low
MSC (LPDDR ctrl + xbar + BT)	0.18	50	medium
LSU (32-inst RISC)	0.10	30	low
TIU (256 B + accumulator)	0.03	15	low
HIF (PCIe Gen3 x1)	0.55	30	medium
SRAM (0.8 MB, 4 banks)	0.71	20	low
LPDDR5X x16 PHY (vendor)	1.20	—	high
Clock + power grid	0.40	—	—
Routing overhead	0.10	—	—
I/O ring + pads + ESD	0.76	—	—
Gross total	4.354	280	
With recommended shrinks	4.014	—	

$d_h \times 2 \times \text{bpv}/8$ formula. Under the ChannelQuant codec of record the bits-per-value is ≈ 4 (measured 4.13–4.38), and the per-model token capacities are pending re-derivation for the Qwen2-1.5B target.

The “32K on-die” claim was wrong. With corrected math, no target model holds 32 K context fully on-die per layer; the 0.4 MB scratchpad holds on the order of 10^2 – 10^3 tokens per layer depending on the KV-head layout. Long contexts serve via LPDDR5X streaming. The exact per-layout capacities and streaming overheads are pending re-derivation for ChannelQuant / Qwen2-1.5B.

MatE’s INT16 accumulator was a real bug. The original spec specified INT16 K-axis accumulators. INT8 $\times$ INT4 produces an 11-bit signed product; reducing $K=128$ requires 18 signed bits, saturating INT16 after approximately 64 accumulations in the worst case. Corrected to INT24 with an INT16 partial-product register inside each PE. The exact margin at the Qwen2-1.5B FFN dimensions is pending re-derivation.

The “3–5 $\times$ compute headroom” claim was a derivation error. In the bandwidth-bound regime, $\text{GOPS}_{\text{needed}} = 2 \cdot \text{BW}_{\text{GB/s/bytes}} / \text{bytes}_{\text{per param}}$, a constant across model sizes (because both numerator and denominator scale with bandwidth). The correct figure is a constant 1.6 $\times$ headroom.

Additional corrections: the “comfortable headroom at PHY = 1.0 mm²” claim was misleading (the sensitivity sweep actually returns +0.002 mm² at 1.0 MB SRAM—exactly fits, no margin); the weight_stream_buffer was 0.05/0.15 MB inconsistent (canonical 0.05); the total SRAM was 0.8/0.85/1.0 MB inconsistent (canonical 0.8). The KV-codec bits-per-value figure that was reconciled at this audit has since been superseded by the ChannelQuant codec of record (≈ 4 bits/value).

None of the corrections changed any architectural decision. Every fix was either a math error or a stale claim from an earlier iteration. We document them here because pre-RTL audits of this kind are, in our experience, frequently skipped in academic chip projects and frequently catch real bugs.

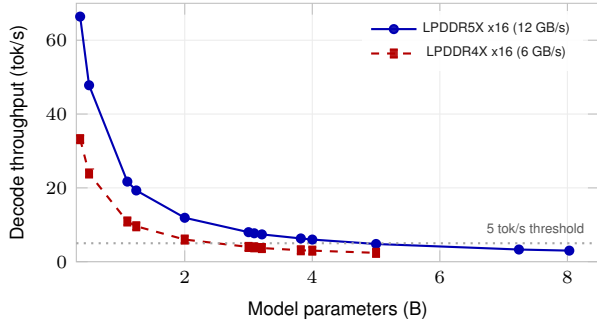


Fig. 3. Decode throughput versus model size under the two PHY options (a bandwidth-bound weight-streaming model, independent of the KV codec). Lambda’s canonical target is up to 1.5B parameters (validated on Qwen2-1.5B), which sits comfortably above the 5 tok/s threshold on the LPDDR5X x16 path with substantial headroom; the curve is drawn out to larger sizes to show where the bandwidth ceiling falls. The LPDDR4X fallback path caps lower (approximately 2.4B parameters above threshold). The larger data points (≥ 3 B) reflect the retired example set and are shown for context only.

VI. PERFORMANCE AND THE LPDDR PHY TRADEOFF

Decode is bandwidth-bound at every reasonable model size. The largest model the chip can serve at a given throughput threshold derives directly from the bandwidth: $\text{params}_{\text{max}} = \text{BW}_{\text{GB/s}} \cdot t_{\text{per tok}} / \text{bytes}_{\text{per param}}$. At LPDDR5X x16 (12 GB/s sustained, $t = 200$ ms for 5 tok/s, 0.5 B/param for W4), the bandwidth ceiling is ≈ 4.8 B parameters. The canonical target (up to 1.5 B, validated on Qwen2-1.5B) sits well inside this ceiling, with substantial throughput headroom. Figure 3 shows the decode throughput curve across a range of model sizes; the larger points (≥ 3 B) reflect the retired example set and are shown for context only.

A. The LPDDR5X-vs-LPDDR4X PHY tradeoff

The dominant uncertainty in Lambda’s area budget is the LPDDR5X x16 PHY footprint at 16 nm, which is NDA-gated and which we estimate at 1.20 mm^2 with $\pm 0.3 \text{ mm}^2$ real swing. The estimate is extrapolated from vendor x64 PHY data points; an authoritative number requires a written Synopsys or Cadence quote in Q2 2026. If the quote returns above 1.35 mm^2 , Lambda’s recommended shrink path no longer closes at 4 mm^2 , and a fallback is required.

The natural fallback is LPDDR4X x16 with a Cadence PHY. LPDDR4X has been shipping at 16 nm in multiple commercial parts since approximately 2018 and has public datasheets, in contrast to LPDDR5X’s NDA-only 16 nm references. The Cadence PHY for LPDDR4X at 16 nm is approximately 1.0 mm^2 with substantially less area variance ($\pm 0.15 \text{ mm}^2$). Adopting LPDDR4X frees approximately 0.2 mm^2 of breathing room and materially de-risks the first-FinFET-PHY integration that is the chip’s largest non-architectural risk.

The cost is bandwidth. LPDDR4X x16 sustains approximately 6 GB/s, halving the bandwidth ceiling from approximately 4.8 B parameters to approximately 2.4 B at the 5 tok/s threshold. The canonical target (Qwen2-1.5B and the ≤ 1.5 B class—e.g. Llama-3.2-1B at 9.6 tok/s, Qwen2.5-1.5B at 8.0 tok/s) remains comfortably above threshold on either PHY, so the LPDDR4X fallback preserves the target class

while trading away only the larger-model headroom. Adopting it keeps a fully public-datasheet PHY at materially lower tape-out risk.

The recommendation in our current planning is to commission both PHY quotes in parallel and gate the decision on the data. If the LPDDR5X x16 quote returns at or below 1.25 mm^2 and a senior FinFET PD engineer is in place, the LPDDR5X path is correct; if the quote exceeds 1.35 mm^2 or the PD-engineer prerequisite is not met, the LPDDR4X path is correct. Neither outcome kills the project.

B. Comparison with prior art

Table II positions Lambda against the closest reference points we are aware of. The mobile-NPU comparison is necessarily qualitative because the Apple Neural Engine [7] and Qualcomm Hexagon NPU architectures are proprietary. Etched.ai’s Sohu transformer ASIC [38] sits at data-center scale on a substantially larger die. The academic-FPGA comparison reflects projects that target similar workloads at substantially larger die areas or older process nodes. Lambda’s distinguishing positions are (i) standalone operation at the small-model (≤ 1.5 B) class without a tightly-coupled SoC, (ii) streaming per-channel KV compression (ChannelQuant) in silicon, (iii) adaptive-precision KV via the TIU, and (iv) open-source academic distribution.

VII. DISCUSSION

A. What we deliberately excluded

Lambda omits several mechanisms that are present in software-side frontier work. We document the exclusions because the omissions are deliberate and reversible in a future revision.

Multi-head Latent Attention (MLA). DeepSeek-V2’s MLA [39] compresses Q/K/V into a low-rank latent space before the head split, reducing per-layer KV by approximately an order of magnitude versus standard MHA. Lambda’s MSC and KVE both assume the conventional $k_h \times d_h$ KV layout. MLA would require a redesign of both blocks; we defer it for a future revision and document the gap explicitly.

Continuous batching and prefix sharing. The single-session chip excludes multi-tenant serving primitives. We exclude them deliberately: a 4 mm^2 die does not have area for the multi-session state, and the on-device deployment target does not benefit from them.

Mixture-of-Experts. MoE routing requires a sparse-dispatch fabric that does not fit our area budget and that is not yet a dominant pattern in the small-model (≤ 1.5 B) class.

Mamba and other SSM blocks. Selective state-space models [40] require different primitive operations from attention; the chip is not architected for them.

Multimodal vision/audio encoders. Out of scope for the chip’s text-only inference target.

Hardware speculative-decoding fabric. Speculative decoding (Medusa [41], EAGLE [42]) and multi-token prediction (DeepSeek-V3 [43]) are architecturally compatible: the host coordinates draft tokens, and the chip runs the verifier pass through a standard batched- N decode schedule. We do not

TABLE II
LAMBDA VERSUS THE CLOSEST ACCELERATOR REFERENCE POINTS.

	Lambda (this work)	Apple ANE*	Qualcomm Hexagon NPU*	Academic FPGA accelerators
Process	TSMC N16FFC	≤N3	≤N4	16/28 nm
Die area	4 mm ² (standalone)	integrated in SoC	integrated in SoC	100+ mm ² (FPGA fabric)
Model class @ 5 tok/s	≤1.5 B params	≥3 B	≥3 B	≤1 B
KV compression	ChannelQuant ≈4 b/val	proprietary	proprietary	FP16 / INT8 (uniform)
Adaptive-precision KV	TIU (per-block)	unknown	unknown	none
Per-channel INT4 KV codec	yes (INT4-FP16 dequant)	unknown	unknown	no
Open source	yes	no	no	varies
Form factor	M.2 2280 carrier	integrated SoC	integrated SoC	dev board
Tape-out path	IMEC mini@sic 2.0	internal	internal	FPGA bitstream (no tape-out)

*Apple ANE and Qualcomm Hexagon NPU architectural details are proprietary; entries reflect publicly disclosed capabilities. Comparison is qualitative.

include a dedicated speculative-decoding accelerator on-die; the LSU schedule is general enough to accommodate the pattern.

B. IP clearance: the Etched patent

Etched.ai’s US patent 2024/0419516 A1 [44] describes an AI hardware platform comprising two structurally separated circuits on a single IC: a systolic array for operations that do not consume past-token data and a self-attention circuit for operations that do. Lambda’s MatE is a *single* 8×8 systolic that mode-multiplexes between weight-stationary (for the no-past-token-data operations the patent describes as belonging to its systolic array) and output-stationary (for the past-token-data operations the patent describes as belonging to its self-attention circuit). The structural separation that grounds the patent’s claim is absent in Lambda’s design; the mode-switch is a CSR bit, not two circuits.

Our reading is that Lambda is outside the claim scope, but a defensive review with our institutional technology transfer office is the appropriate next step before silicon commit. The patent does highlight a real scheduling distinction we adopt explicitly: the LSU schedule treats weight-matmul tiles and attention-matmul tiles as separate dispatch primitives with separate optimization targets (K-axis size, operand stationarity).

C. Risks and open questions

The largest non-architectural risk is the LPDDR5X x16 PHY integration: this is the team’s first FinFET-era PHY tape-out, and the 1.20 mm² estimate has ±0.3 mm² real swing. Mitigation is the vendor quote (Q2 2026) and a senior FinFET physical-design engineer in place by Q3 2026; the LPDDR4X fallback (Section VI) is the documented release valve.

The IMEC mini@sic 2.0 actual pricing for 4 mm² N16FFC is non-public; the \$60–100 K range is an academic-discount-tier estimate. A direct quote via `eptsmc@imec.be` is the first concrete action item on the project’s gantt.

VIII. CONCLUSION

We have presented Lambda, a 4 mm² open-source academic transformer-decode ASIC at TSMC N16FFC that serves the small-model (≤1.5 B, validated on Qwen2-1.5B) on-device deployment regime at 6–8 tok/s in 2.6 W. The chip integrates

a streaming silicon implementation of *ChannelQuant*—per-channel INT4 keys, per-token INT4 values, and a static top-*k* FP16 outlier-channel lane, following the KIVI [1] / KVQuant [2] recipe—together with the first silicon implementation of adaptive-precision KV cache management via on-die attention-entropy accumulation (the TIU), in an open-source academic standalone accelerator. We have documented the design-space exploration, the LPDDR5X-versus-LPDDR4X PHY tradeoff, and an explicit pre-RTL audit. The KV-compression block’s physical PPA at N16FFC (area, power, $F_{\max}$), and the target-model throughput and capacity figures, are pending re-measurement and re-derivation for ChannelQuant on Qwen2-1.5B. The next phase is high-level synthesis of the MatE processing element and the KVE ChannelQuant datapath—the two long poles. The full architecture specification, the audit log, the literature review, and the per-block HLS scaffolding are publicly available at github.com/LonghornSilicon/architecture.

ACKNOWLEDGMENTS

The author thanks the UT Austin Computer Architecture Lab, the IMEC / Europractice mini@sic 2.0 program, and the broader open-source quantization research community whose work this chip integrates.

REFERENCES

- [1] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “KIVI: A tuning-free asymmetric 2bit quantization for KV cache,” in *International Conference on Machine Learning (ICML)*, 2024, arXiv:2402.02750. Recipe basis for Lambda’s ChannelQuant KV codec (per-channel keys, per-token values). Author list and arXiv ID taken from the published paper; verify before camera-ready.
- [2] C. Hooper, S. Kim, H. Mohammadi, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2401.18079. Recipe basis for Lambda’s ChannelQuant KV codec (per-channel + dense-and-sparse outlier handling).
- [3] Qwen Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024, qwen2.5-3B / -1.5B; 2 KV-head layout enables Lambda’s long-context champion mode.
- [4] Meta AI, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024, llama-3.2-3B / -1B technical report; primary demo target family.
- [5] Mistral AI and NVIDIA, “Mistral-NeMo: A multilingual open-weight model,” Mistral AI / NVIDIA joint release, 2024, mistral-NeMo-3B is one of Lambda’s primary demo targets.

- [6] Microsoft Research, “Phi-3 technical report: A highly capable language model locally on your phone,” *arXiv preprint arXiv:2404.14219*, 2024, phi-3.5-mini; 32 KV-head layout is the KV-heavy outlier in Lambda’s target set.
- [7] Apple Machine Learning Research, “Apple intelligence foundation language models,” *arXiv preprint arXiv:2407.21075*, 2024, on-device 3B-class deployment story directly comparable to Lambda’s target.
- [8] NVIDIA Corporation, “NVIDIA Ampere GA102 GPU architecture whitepaper,” NVIDIA technical whitepaper, 2020, reference GPU tensor-core dataflow; Lambda’s compressed-domain attention contrasts with the FP16/FP8 tensor-core pattern.
- [9] —, “NVIDIA H100 tensor core GPU architecture,” NVIDIA technical whitepaper, 2022.
- [10] —, “NVIDIA Blackwell architecture and the NVFP4 format,” NVIDIA developer documentation, 2024, cited for low-bit numeric-format context (NVFP4 / Blackwell).
- [11] IMEC and Europractice, “EUROPRACTICE mini@sic 2.0 service for TSMC technologies,” IMEC / Europractice academic-shuttle program announcement, 2025.
- [12] “Adaptive KV-cache quantization for lightweight on-device LLMs,” *arXiv preprint arXiv:2604.04722*, 2026.
- [13] Z. Zhang *et al.*, “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” in *NeurIPS*, 2023, arXiv:2306.14048.
- [14] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, arXiv:2205.14135.
- [15] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” *arXiv preprint arXiv:2307.08691*, 2023.
- [16] J. Shah *et al.*, “FlashAttention-3: Fast and accurate attention with asynchrony and low-precision,” *arXiv preprint arXiv:2407.08608*, 2024.
- [17] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *arXiv preprint arXiv:2104.09864*, 2021, the RoPE positional encoding scheme used by Llama, Mistral, Qwen, and most modern LLMs.
- [18] B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, arXiv:1910.07467.
- [19] N. Shazeer, “GLU variants improve transformer,” *arXiv preprint arXiv:2002.05202*, 2020, introduces SwiGLU, the FFN activation used by Llama-3 family.
- [20] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *ACM Symposium on Operating Systems Principles (SOSP)*, 2023, arXiv:2309.06180.
- [21] Z. Ye *et al.*, “FlashInfer: Efficient and customizable attention engine for LLM inference serving,” *arXiv preprint arXiv:2501.01005*, 2025.
- [22] E. Kurtic *et al.*, “Give me BF16 or give me death: Accuracy-performance trade-offs in LLM quantization,” *arXiv preprint arXiv:2411.02355*, 2024.
- [23] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Int’l Conf. on Learning Representations (ICLR)*, 2023, arXiv:2210.17323.
- [24] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Conf. on Machine Learning and Systems (MLSys)*, 2024, arXiv:2306.00978.
- [25] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *Int’l Conf. on Machine Learning (ICML)*, 2023, arXiv:2211.10438.
- [26] Survey Authors, “A survey on KV-cache acceleration for large language model inference,” *arXiv preprint arXiv:2506.13131*, 2025.
- [27] S. Ashkboos *et al.*, “QuaRot: Outlier-free 4-bit inference in rotated LLMs,” *arXiv preprint arXiv:2404.00456*, 2024.
- [28] Various Authors, “RotateKV: Accurate and robust 2-bit KV cache quantization via outlier-aware adaptive rotations,” *arXiv preprint arXiv:2401.18079*, 2024, cited alongside QuaRot/TurboQuant in the rotation-based quant family.
- [29] S. Ashkboos *et al.*, “TurboQuant: Optimal scalar codebook KV compression,” *arXiv preprint arXiv:2504.19874*, 2025, accepted to ICLR 2026. Cited as prior work only; not Lambda’s codec of record (which is ChannelQuant, following KIVI/KVQuant).
- [30] C. Talasila, “Adaptive-precision attention: Precision controller and MAC array specifications,” LonghornSilicon GitHub project (companion to this work), 2026, github.com/LonghornSilicon/adaptive-precision-attention.
- [31] Z. Liu *et al.*, “Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time,” in *NeurIPS*, 2023, arXiv:2305.17118.
- [32] M. Oren *et al.*, “TOVA: Transformers are multi-state RNNs with token omission via attention,” *arXiv preprint arXiv:2401.06104*, 2024.
- [33] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” *arXiv preprint arXiv:2309.17453*, 2023, grounds the `tiu_streaming_llm` eviction mode.
- [34] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proc. Int’l Symp. Computer Architecture (ISCA)*, 2017, arXiv:1704.04760.
- [35] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan *et al.*, “TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings,” in *Proc. Int’l Symp. Computer Architecture (ISCA)*, 2023, reference for systolic-array dataflow at scale; informs MatE’s weight-stationary primary mode.
- [36] Y. Lin, S. Dhar, W. Li, H. Ren, B. Khailany, and D. Z. Pan, “DREAM-Place: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement,” in *ACM/IEEE Design Automation Conference (DAC)*, 2019, arXiv:1809.09457.
- [37] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, “MLIR: Scaling compiler infrastructure for domain specific computation,” in *IEEE/ACM Int’l Symp. on Code Generation and Optimization (CGO)*, 2021, arXiv:2002.11054.
- [38] Etched.ai, “Sohu: The first transformer ASIC (hotchips 2024 presentation),” HotChips 2024 industrial session, 2024, closest commercial reference point for a transformer-specific ASIC; data-center-scale, not directly comparable to Lambda’s 4 mm² academic target.
- [39] DeepSeek-AI, “DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model,” *arXiv preprint arXiv:2405.04434*, 2024.
- [40] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [41] T. Cai *et al.*, “Medusa: Simple LLM inference acceleration framework with multiple decoding heads,” *arXiv preprint arXiv:2401.10774*, 2024.
- [42] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE: Speculative sampling requires rethinking feature uncertainty,” *arXiv preprint arXiv:2401.15077*, 2024.
- [43] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024, introduces multi-token prediction (MTP) at scale.
- [44] G. Uberti and C. Zhu, “Parallel execution of self-attention-based AI models,” U.S. Patent Application 2024/0419516 A1, Etched.ai, Inc., 2024.